Topology-aware Prime-Ends — Algorithm, Video-Buffer Extension & Read/Write System

Level: Design + implementation recipe (ready to implement)

Purpose: - Produce a topologically-faithful discrete prime-ends construction for 2D image patches (and tiled surface patches) that: (a) captures distinct approach classes to boundary vertices; (b) uses topology signals (genus, Betti, turning numbers, Euler) to guide sampling and multiplicity decisions; (c) works frame-by-frame on a video storage buffer (image bitmaps) with temporal coherence; and (d) operates as a read/write-enabled system that both *hypothesizes* an underlying 3D surface (from 2D observations) and *renders* visualizations/exports.

---

1. High-level summary

1. **Discrete prime-ends via nested cross-cut chains.** We generate many short interior cross-cuts (simple paths between boundary points), group them into nested chains that shrink toward boundary locations, and cluster chains into prime-end equivalence classes via mutual eventual containment.
2. **Topology-aware guidance.** Use Betti numbers, genus, Euler characteristic, turning numbers and curvature concentrations to: allocate sampling budgets, prefer cross-cuts near high-curvature/vertex regions, and expect multiplicities near holes or necks.
3. **Video-buffer extension.** Operate on a **storage buffer of image bitmaps** (instead of processing live video frames directly): load or stream bitmaps from storage, compute per-bitmap prime-ends, use optional temporal metadata (timestamps, frame indices) or precomputed optical-flow/feature-tracks for temporal linking, and maintain temporal smoothing and homology-persistence to stabilize prime-end labels.
4. **Read/Write-enabled system.** Provide a modular service with persistent storage for topological state, prime-end histories, 3D hypotheses and render artifacts. Support incremental write (append-only change logs) and read queries for visualization, downstream analysis, or reconstruction.

---

2. Algorithm (concise step-by-step)

Inputs

- `I`: image or video frame (grayscale/color). If video, frames stored in a storage buffer (image bitmaps).
- `B`: detected boundary chain (ordered boundary pixels) for the patch.
- optional topology seeds: approximate `b0,b1` (Betti), `χ` (Euler), `genus`, turning-number map, curvature map.
- params: sampling budget `S`, vertex window `w_v`, collapse radius `ε_col`, nesting threshold `τ_nest`.

Outputs

- `PE`: set of prime ends {PE_i} with metadata (representative cross-cuts, boundary anchor, multiplicity)
- per-pixel mapping `pixel → PE_id` (soft/hard)
- boundary parameterization with stretched/quotiented coordinates
- temporal links (for video): `PE_t ↔ PE_{t+1}`

Core steps (per frame)

1. **Preprocess**: boundary extraction (Canny or edge-follow), polygonal simplify to locate candidate vertices `V`; build interior pixel graph `G` (4/8-neighbor) with weights `1 + λ·gradMag`.
2. **Topology estimation**: compute connected components and an approximation to `b0,b1` (fast union-find + cycle detection). If available, use external topology priors to set `S` and `p_v` (prob. prefer vertices).
3. **Cross-cut sampling** (budget S): sample endpoint pairs `(p,q) ∈ B×B` with vertex bias; compute simple shortest-path `C = SP(G,p,q)` (Dijkstra/A*). Record only cuts that separate a small region near some target vertex (via quick flood-fill) or otherwise are topologically informative.
4. **Build chains**: group cuts that isolate the same vertex—sort by enclosed area and form nested subsequences `C_1 ⊃ C_2 ⊃ ...`.
5. **Chain equivalence & clustering**: mark two chains equivalent if each eventually contains cuts of the other (discrete nesting test). Agglomerative clustering by nesting/containment yields `PE` classes.
6. **Pixel assignment**: for each pixel, choose the smallest-area chain containing it and label it with that chain's prime-end ID (fallback: outer class).
7. **Boundary parameterization**: map each `PE` to an angular interval on a model boundary circle. Width ∝ inverse mean distance-to-anchor (smaller = finer). At vertices apply collapse + multiplicity rules (map small ε_col neighborhoods to singletons optionally while allocating multiple tiny ticks for multiple prime-ends).
8. **Temporal linking (video)**: match `PE_t` → `PE_{t-1}` by IoU of pixel support, chain similarity, and optical-flow traced correspondences. Stabilize labels via an exponential moving average and homology-persistence scoring.

---

3. Topology-aware refinements (details)

- **Sampling budget adaptivity:** `S = S0·(1 + α·b1 + β·curvature_peaks)`.
- **Homology-guided cross-cuts:** if `b1>0`, generate ring-cross-cuts encircling basis cycles to ensure hole-mouth prime-ends are sampled.
- **Turning-number steering:** treat peaks of turning-number as vertex seeds; allocate dense sampling windows `w_v` around them.
- **Nesting strictness:** use area ratio test `area(C_{i+1})/area(C_i) < τ_nest` to validate nestedness; allow small violations due to discretization but require monotone shrinkage on average.

---

4. Video-buffer specifics

- **Storage buffer semantics (image bitmaps):** store last `N` frames with frame index. Each frame record: `B`, `crosscuts`, `chains`, `PE` labels, pixel assignments, optional homology summary.
- **Incremental update strategy:** only recompute full cross-cut sampling in regions that changed more than a threshold (per-pixel difference or flow magnitude). For minor motion use label propagation.
- **Temporal matching:** use 3 cues to match PE across frames: (1) pixel-support IoU; (2) chain-shape similarity (Hausdorff); (3) tracked anchor points via optical flow. Combine into score and link mappings above threshold.
- **Homology persistence across time:** maintain persistence pairs for topological features; remove ephemeral prime-ends that do not persist beyond length `T_min` frames.

---

5. Read/Write-enabled system architecture

Modules 1. **Input Manager** — ingests images/frames, normalizes, places them in storage buffer (image bitmaps). 2. **Preprocessor** — edge detection, vertex detection, gradient map. 3. **Topology Engine** — computes/updates Betti numbers, cycle bases, curvature/turning maps. 4. **Cross-cut Sampler** — produces candidate cuts (parallelizable). Writes cuts to local store. 5. **Chain Manager** — constructs chains, tests nesting, clusters to prime-ends. 6. **Hypothesis Engine** — builds 3D-surface hypotheses (see §6) using prime-end info + silhouettes + multi-frame cues. 7. **Renderer** — visualizes prime-ends, stretched boundary mapping, 3D hypothesized meshes (OBJ/PLY/GLTF exports). 8. **Storage & API** — persistent DB (object store + metadata DB) exposing read/write endpoints.

Data model / persistable objects - `FrameRecord = {frame_id, timestamp, B, vertices, crosscuts[], chains[], PE[], pixel_map}` - `PrimeEnd = {id, representative_chains, anchors, pixel_support_bitmap, birth_frame, last_seen, persistence_score}` - `TopologySummary = {b0,b1,chi,cycle_basis}` - `Hypothesis3D = {mesh (PLY/OBJ), genus, homology_summary, confidence, provenance}`

Read/Write semantics - **Append-only** event log for frame records and PE births/deaths; idempotent writes allowed. Use sequence numbers to prevent races. - **Mutable metadata store** for hypotheses & rendered artifacts (with versioning). Allow read queries for historical frames and current state. - **Transactions**: write operations that update multiple tables (e.g., linking PE across frames) should be transactional or use optimistic concurrency with conflict resolution.

APIs (examples) - `POST /frames` — upload frame (returns frame_id) - `GET /frames/{id}/prime-ends` — returns PE list + pixel support mask - `GET /prime-ends/{id}` — returns history, representative chains, and anchor locations - `POST /hypotheses` — request 3D reconstruction from a frame range and options (e.g., enforce genus=g) - `GET /hypotheses/{id}/mesh` — download OBJ/PLY/GLTF

---

6. Hypothesizing 3D surface from prime-end data

Why prime-ends help: multiplicities at boundary points, adjacency of prime ends, and pixel support patterns encode accessibility and “sides” of projected surface features (tips, necks, tunnels) — these are topology cues for 3D reconstruction.

Pipeline options 1. **Single-view + priors**: combine silhouette, shading cues and prime-end adjacency to fit a parametric template (e.g., deformable genus-constrained mesh) via energy minimization that enforces observed topology (target genus/b1) and boundary accessibility constraints coming from prime-end multiplicities. 2. **Multi-frame (video)**: use tracked prime-ends across frames to produce multi-view silhouettes and sparse correspondences; run volumetric reconstruction (space carving, visual hull) or multi-view stereo to obtain surfaces. Use homology constraints to merge/reject topological hypotheses. 3. **Implicit function + topology constraints**: solve for an implicit scalar field whose 0-level set matches projection constraints and whose topology (genus) is regularized by penalties derived from prime-end homology signals.

Implementation notes - Use prime-end adjacency graph to initialize mesh connectivity (e.g., create loops across prime-end anchors and stitch interior). Enforce genus by adding handles where prime-end pairings indicate tunnels. - Confidence scoring: use persistence score from video and homology matching to accept/reject 3D hypotheses.

---

7. Visualization & rendering

- 2D: per-pixel prime-end heatmap; stretched boundary circle with angular intervals; vertex collapse ticks & multiplicity markers.
- 3D: project prime-end anchors to sphere or mesh UVs; color mesh by inferred prime-end mapping; export scenes as GLTF for interactive viewing.

---

8. Implementation tips, libraries & complexity

- **Edge/vertex detection:** OpenCV (Canny, findContours), shapely for polygon ops.
- **Graph & shortest paths:** custom grid-based Dijkstra/A* (pygrap or NetworkX for prototypes; C++ for speed). Use heuristics to limit ROI.
- **Homology:** use `gudhi`, `ripser.py`, or `phat` for simple cycle-basis / persistence tests.
- **Optical flow / tracking:** OpenCV (Farneback, Pyramidal Lucas-Kanade) or RAFT for robust flows.
- **3D reconstruction:** Open3D, OpenMVG/OpenMVS, PoissonRecon, or custom implicit solvers.

Complexity: cross-cut sampling dominates ($O(S \cdot \text{cost}(SP))$). Use parallel paths, A* with admissible heuristics, and local ROI to keep runtime practical. Temporal coherence avoids full recompute each frame.

---

9. Example workflows

1. **One-off analysis:** single frame → run full pipeline → export PE map + 3D hypothesis (single-view template-fit).

2. **Video analysis:** continuous ingestion → incremental PE updates → periodic 3D reconstruction requests across selected frame windows → store hypotheses and render.
3. **Interactive mode:** user marks expected genus or toggles collapse radius; pipeline updates prime-ends in real-time for a storage buffer (image bitmaps).

---

10. Next steps & extensions

- Add learning-based cross-cut prioritizer (train a small model to suggest valuable endpoint pairs using supervised examples of prime-ends).
- Integrate mesh-aware shortest-path (geodesics on hypothesized mesh) when partial 3D hypotheses exist, for tighter chain construction.
- Offer GPU-accelerated graph kernels for high-res frames.

---

11. Appendix — key pseudocode (compact)

```
for each frame in storage_buffer:
    B, V = extract_boundary_and_vertices(frame)
    G = build_pixel_graph(frame)
    topology = estimate_topology(G)
    S = adapt_budget(base_S, topology)
    crosscuts = sample_crosscuts(G, B, V, S)
    chains = form_nested_chains(crosscuts)
    prime_ends = cluster_chains(chains)
    pixel_map = assign_pixels_to_prime_ends(G, prime_ends, chains)
    persist_frame_record(frame_id, B, V, crosscuts, chains, prime_ends,
pixel_map)
    if is_video:
        match_prime_ends_temporally(prev_frame, current_frame)
    if reconstruction_requested:
        hypothesis = build_3D_hypothesis(prime_ends_history)
        persist_hypothesis(hypothesis)
```

If you want, I can: - produce a runnable Python notebook implementing a **minimal working prototype** for the rectangle experiment (edge detection → cross-cut sampling → chain clustering → visualization) that you can run locally, or - expand any section of this document into a more detailed design doc (API definitions, DB schema, or production architecture), or - generate sample input/output JSON schemas for the read/write API.

Tell me which one you want next and I will produce it.